

Database Normalization to BCNF

MusicDashboard Application Schema

Sebastian Dorata

Subject: Database Design & Normalization

Method: Boyce–Codd Normal Form (BCNF)

Date: February 10, 2026

All functional dependencies derived from entity source code.
Attribute closure computed via the standard Armstrong Axioms.

Contents

1	Background and Methodology	3
1.1	The Source Schema	3
1.2	Normal Form Definitions	3
1.2.1	Functional Dependency (FD)	3
1.2.2	Attribute Closure X^+	3
1.2.3	Superkey and Candidate Key	3
1.2.4	Boyce–Codd Normal Form (BCNF)	3
1.3	Decomposition Algorithm	4
2	Universal Relation and Denormalized Schema	5
3	Attribute Closure Computations	6
3.1	Closure of <code>user_id</code>	6
3.2	Closure of <code>email</code>	6
3.3	Closure of <code>song_id</code>	6
3.4	Closure of <code>album_id</code>	6
3.5	Closure of <code>artist_id</code>	6
3.6	Closure of <code>playback_id</code>	7
3.7	Closure of <code>{user_id, report_year, week_of_year}</code>	7
3.8	Closure of <code>{user_id, song_id}</code> in Favourites	7
4	Attribute Closure Computations	8
4.1	Closure of <code>user_id</code>	8
4.2	Closure of <code>email</code>	8
4.3	Closure of <code>song_id</code>	8
4.4	Closure of <code>album_id</code>	8
4.5	Closure of <code>artist_id</code>	9
4.6	Closure of <code>playback_id</code>	9
4.7	Closure of <code>{user_id, report_year, week_of_year}</code>	9
4.8	Closure of <code>{user_id, song_id}</code> in Favourites	9
5	BCNF Violation Analysis on Universal Relation U	10
6	Step-by-Step Decomposition to BCNF	11
6.1	Step 1 — Extract Users	11
6.2	Step 2 — Extract Albums	11
6.3	Step 3 — Extract Songs	11
6.3.1	Check: Was FD18 (<code>song_id</code> $\rightarrow$ <code>album_title</code>) eliminated?	12
6.4	Step 4 — Extract Artists	12
6.5	Step 5 — Extract Genres	12
6.6	Step 6 — Extract PlaybackHistory	12
6.7	Step 7 — Extract Playlists	13
6.8	Step 8 — Extract Favourites	13
6.9	Step 9 — Extract WeeklyReports	13
7	Join / Associative Tables	14

8	Final BCNF Schema	15
8.1	Complete Relation Listing	15
8.2	BCNF Verification Summary	16
9	Summary of Decomposition Steps	17
10	Notable Design Observations	18
10.1	Relations with Multiple Candidate Keys	18
10.2	Transitive Dependencies Eliminated	18
10.3	FD Preservation	18
10.4	Lossless-Join Property	18

Background and Methodology

The Source Schema

The MusicDashboard application uses a Spring Boot / JPA back-end with the following nine core entity tables:

Entity	Purpose
Users	Application accounts
Songs	Audio tracks with metadata
Albums	Album records with artwork
Artists	Music artists
Genres	Genre taxonomy
PlaybackHistory	Per-play event log
Playlists	User-created playlists
Favourites	User song bookmarks
WeeklyReports	Pre-computed weekly stats

Three additional tables arise purely from many-to-many join tables in JPA (`song_artists`, `song_genres`, `album_artists`, `playlist_songs`) and contain only foreign-key pairs; they are already in BCNF by construction and are addressed briefly in Section 7.

Normal Form Definitions

Functional Dependency (FD)

A functional dependency $X \rightarrow Y$ holds in relation R if and only if for every pair of tuples $t_1, t_2 \in R$:

$$t_1[X] = t_2[X] \implies t_1[Y] = t_2[Y]$$

That is, the value of X uniquely determines the value of Y .

Attribute Closure X^+

The closure of attribute set X under a set of FDs F is the largest set X^+ such that $X \rightarrow X^+$ is derivable from F using Armstrong's Axioms (Reflexivity, Augmentation, Transitivity).

Superkey and Candidate Key

X is a superkey of R iff X^+ contains all attributes of R . A candidate key is a minimal superkey.

Boyce–Codd Normal Form (BCNF)

Relation R is in BCNF with respect to a set of FDs F if and only if for every non-trivial FD $X \rightarrow Y$ that holds in R :

$$X \text{ is a superkey of } R$$

Equivalently, the only functional dependencies allowed are those where the left-hand side contains a candidate key.

Decomposition Algorithm

Given a relation R with FD set F that violates BCNF due to $X \rightarrow Y$ (where X is not a superkey):

1. Compute X^+ using F .
2. Decompose R into:
 - $R_1 = X^+$ (contains the violating FD)
 - $R_2 = X \cup (R \setminus X^+)$ (remainder with X as join key)
3. Project F onto R_1 and R_2 , then repeat for each sub-relation.

This process is lossless-join by construction (the shared attribute set X is a key of R_1), though it may not always preserve every FD.

Universal Relation and Denormalized Schema

We begin by merging all entity tables into a single Universal Relation U to make every functional dependency explicit before decomposing.

$$\begin{aligned}
 U = \{ & \underbrace{\text{user_id, email, username, password_hash, user_created_at, last_login,}}_{\text{user attrs}} \\
 & \underbrace{\text{song_id, song_title, file_path, duration, file_format, bit_rate, listen_count, track_num,}}_{\text{song attrs}} \\
 & \underbrace{\text{album_id, album_title, release_year, album_art_path,}}_{\text{album attrs}} \underbrace{\text{artist_id, artist_name, bio,}}_{\text{artist attrs}} \\
 & \underbrace{\text{genre_id, genre_name,}}_{\text{genre attrs}} \underbrace{\text{playback_id, played_at, duration_played, completed, source,}}_{\text{playback attrs}} \\
 & \underbrace{\text{playlist_id, playlist_name, description, is_public, playlist_created_at,}}_{\text{playlist attrs}} \\
 & \underbrace{\text{favourite_id, favourited_at,}}_{\text{favourite attrs}} \\
 & \underbrace{\text{report_id, report_year, week_of_year, total_songs_played, total_minutes}}_{\text{report attrs}}
 \end{aligned}$$

The total attribute count of U is **38**. A single tuple in U represents: a user who played a song (from an album, by an artist, of a genre), which was logged in the playback history, favourited, added to a playlist, and summarised in a weekly report. This produces enormous redundancy, which the normalization process below eliminates.

Attribute Closure Computations

We compute X^+ for every candidate key and every left-hand side appearing in F , to confirm superkey status and detect BCNF violations in U .

Closure of `user_id`

Iteration	Closure so far	FD applied
Start	{ <code>user_id</code> }	FD1
Apply FD1	+ { <code>email</code> , <code>username</code> , <code>password_hash</code> , <code>user_created_at</code> , <code>last_login</code> }	
Fixed point	{ <code>user_id</code> , <code>email</code> , <code>username</code> , <code>password_hash</code> , <code>user_created_at</code> , <code>last_login</code> }	

Closure of `email`

Iteration	Closure so far	FD applied
Start	{ <code>email</code> }	FD2
Apply FD2	+ { <code>user_id</code> , <code>username</code> , <code>password_hash</code> , <code>user_created_at</code> , <code>last_login</code> }	
Fixed point	all <i>Users</i> attributes	

$\text{closure}(\text{email}) = \text{all } Users \text{ attributes.} \Rightarrow \text{Alternate candidate key of } Users.$

Closure of `song_id`

Iteration	Closure so far	FD applied
Start	{ <code>song_id</code> }	FD4
Apply FD4	+ { <code>song_title</code> , <code>file_path</code> , <code>duration</code> , <code>file_format</code> , <code>bit_rate</code> , <code>listen_count</code> , <code>track_num</code> }	
Apply FD6	+ { <code>album_title</code> , <code>release_year</code> , <code>album_art_path</code> }	FD6
Fixed point	all <i>Songs</i> and <i>Albums</i> attributes	

$\text{closure}(\text{song_id}) = \text{all } Songs \text{ attributes plus all } Albums \text{ attributes.}$

$\Rightarrow \text{Superkey of } Songs. \text{ Candidate key: } \{\text{song_id}\}. \text{ Also derives album attributes transitively via } \text{album_id}.$

Closure of `album_id`

Iteration	Closure so far	FD applied
Start	{ <code>album_id</code> }	FD6
Apply FD6	+ { <code>album_title</code> , <code>release_year</code> , <code>album_art_path</code> }	
Fixed point	{ <code>album_id</code> , <code>album_title</code> , <code>release_year</code> , <code>album_art_path</code> }	

$\text{closure}(\text{album_id}) = \text{all } Albums \text{ attributes.} \Rightarrow \text{Candidate key of } Albums.$

Closure of `artist_id`

Iteration	Closure so far	FD applied
Start	{ <code>artist_id</code> }	FD8
Apply FD8	+ { <code>artist_name</code> , <code>bio</code> }	
Fixed point	{ <code>artist_id</code> , <code>artist_name</code> , <code>bio</code> }	

$\text{closure}(\text{artist_id}) = \text{all } \textit{Artists} \text{ attributes.} \Rightarrow \text{Candidate key of } \textit{Artists}.$

Closure of `playback_id`

Iteration	Closure so far	FD applied
Start	{ <code>playback_id</code> }	FD12
Apply FD12	+ { <code>user_id</code> , <code>song_id</code> , <code>played_at</code> , <code>duration_played</code> , <code>completed</code> , <code>source</code> }	
Apply FD1	+ { <code>email</code> , <code>username</code> , <code>password_hash</code> , <code>user_created_at</code> , <code>last_login</code> }	FD1
Apply FD4	+ { <code>song_title</code> , <code>file_path</code> , <code>duration</code> , <code>file_format</code> , <code>bit_rate</code> , <code>listen_count</code> , <code>track_num</code> , <code>album_id</code> }	FD4
Apply FD6	+ { <code>album_title</code> , <code>release_year</code> , <code>album_art_path</code> }	FD6
Fixed point	all reachable attributes	

$\text{closure}(\text{playback_id})$ covers all attributes reachable from a playback event.

$\Rightarrow$ Candidate key of *PlaybackHistory*.

The transitive reach is large—confirming that the Universal Relation violates BCNF badly.

Closure of {`user_id`, `report_year`, `week_of_year`}

Iteration	Closure so far	FD applied
Start	{ <code>user_id</code> , <code>report_year</code> , <code>week_of_year</code> }	
Apply FD17	+ { <code>report_id</code> , <code>total_songs_played</code> , <code>total_minutes</code> }	FD17
Apply FD1	+ { <code>email</code> , <code>username</code> , <code>password_hash</code> , <code>user_created_at</code> , <code>last_login</code> }	FD1
Fixed point	all <i>WeeklyReports</i> and <i>Users</i> attributes	

$\text{closure}(\{\text{user_id}, \text{report_year}, \text{week_of_year}\}) = \text{all } \textit{WeeklyReports} \text{ attributes plus } \textit{Users}.$

$\Rightarrow$ Composite candidate key of *WeeklyReports*.

Closure of {`user_id`, `song_id`} in *Favourites*

Iteration	Closure so far	FD applied
Start	{ <code>user_id</code> , <code>song_id</code> }	
Apply FD15	+ { <code>favourite_id</code> , <code>favourited_at</code> }	FD15
Fixed point	{ <code>user_id</code> , <code>song_id</code> , <code>favourite_id</code> , <code>favourited_at</code> }	

$\text{closure}(\{\text{user_id}, \text{song_id}\}) = \text{all } \textit{Favourites} \text{ attributes}.$

$\Rightarrow$ Composite alternate candidate key of *Favourites*.

Attribute Closure Computations

We compute X^+ for every candidate key and every left-hand side appearing in F , to confirm superkey status and detect BCNF violations in U .

Closure of `user_id`

Iteration	Closure so far	FD applied
Start	{ <code>user_id</code> }	
Apply FD1	+ { <code>email</code> , <code>username</code> , <code>password_hash</code> , <code>user_created_at</code> , <code>last_login</code> }	FD1
Fixed point	{ <code>user_id</code> , <code>email</code> , <code>username</code> , <code>password_hash</code> , <code>user_created_at</code> , <code>last_login</code> }	

$\{\text{user_id}\}^+ = \{\text{user_id}, \text{email}, \text{username}, \text{password_hash}, \text{user_created_at}, \text{last_login}\}$
 $\Rightarrow$ Superkey of *Users*. Candidate key: {`user_id`}.

Closure of `email`

Iteration	Closure so far	FD applied
Start	{ <code>email</code> }	
Apply FD2	+ { <code>user_id</code> , <code>username</code> , <code>password_hash</code> , <code>user_created_at</code> , <code>last_login</code> }	FD2
Fixed point	all <i>Users</i> attributes	

$\{\text{email}\}^+ = \text{all } \textit{Users} \text{ attributes. } \Rightarrow$ Alternate candidate key of *Users*.

Closure of `song_id`

Iteration	Closure so far	FD applied
Start	{ <code>song_id</code> }	
Apply FD4	+ { <code>song_title</code> , <code>file_path</code> , <code>duration</code> , <code>file_format</code> , <code>bit_rate</code> , <code>listen_count</code> , <code>track_num</code> }	FD4
Apply FD6	+ { <code>album_title</code> , <code>release_year</code> , <code>album_art_path</code> }	FD6
Fixed point	all <i>Songs</i> and <i>Albums</i> attributes	

$\{\text{song_id}\}^+ = \text{all } \textit{Songs} \text{ attributes plus all } \textit{Albums} \text{ attributes.}$

$\Rightarrow$ Superkey of *Songs*. Candidate key: {`song_id`}. Also derives album attributes transitively via `album_id`.

Closure of `album_id`

Iteration	Closure so far	FD applied
Start	{ <code>album_id</code> }	
Apply FD6	+ { <code>album_title</code> , <code>release_year</code> , <code>album_art_path</code> }	FD6
Fixed point	{ <code>album_id</code> , <code>album_title</code> , <code>release_year</code> , <code>album_art_path</code> }	

$\{\text{album_id}\}^+ = \text{all } \textit{Albums} \text{ attributes.} \Rightarrow \text{Candidate key of } \textit{Albums}.$

Closure of artist_id

Iteration	Closure so far	FD applied
Start	$\{\text{artist_id}\}$	
Apply FD8	$+ \{\text{artist_name}, \text{bio}\}$	FD8
Fixed point	$\{\text{artist_id}, \text{artist_name}, \text{bio}\}$	

$\{\text{artist_id}\}^+ = \text{all } \textit{Artists} \text{ attributes.} \Rightarrow \text{Candidate key of } \textit{Artists}.$

Closure of playback_id

Iteration	Closure so far	FD applied
Start	$\{\text{playback_id}\}$	FD12
Apply FD12	$+ \{\text{user_id}, \text{song_id}, \text{played_at}, \text{duration_played}, \text{completed}, \text{source}\}$	
Apply FD1	$+ \{\text{email}, \text{username}, \text{password_hash}, \text{user_created_at}, \text{last_login}\}$	FD1
Apply FD4	$+ \{\text{song_title}, \text{file_path}, \text{duration}, \text{file_format}, \text{bit_rate}, \text{listen_count}, \text{track_num}, \text{album_id}\}$	FD4
Apply FD6	$+ \{\text{album_title}, \text{release_year}, \text{album_art_path}\}$	FD6
Fixed point	all reachable attributes	

$\{\text{playback_id}\}^+$ covers all attributes reachable from a playback event.

$\Rightarrow$ Candidate key of *PlaybackHistory*. *The transitive reach is large—confirming that the Universal Relation violates BCNF badly.*

Closure of $\{\text{user_id}, \text{report_year}, \text{week_of_year}\}$

Iteration	Closure so far	FD applied
Start	$\{\text{user_id}, \text{report_year}, \text{week_of_year}\}$	FD17
Apply FD17	$+ \{\text{report_id}, \text{total_songs_played}, \text{total_minutes}\}$	
Apply FD1	$+ \{\text{email}, \text{username}, \text{password_hash}, \text{user_created_at}, \text{last_login}\}$	FD1
Fixed point	all <i>WeeklyReports</i> and <i>Users</i> attributes	

$\{\{\text{user_id}, \text{report_year}, \text{week_of_year}\}\}^+ = \text{all } \textit{WeeklyReports} \text{ attributes plus } \textit{Users}.$

$\Rightarrow$ Composite candidate key of *WeeklyReports*.

Closure of $\{\text{user_id}, \text{song_id}\}$ in Favourites

Iteration	Closure so far	FD applied
Start	$\{\text{user_id}, \text{song_id}\}$	FD15
Apply FD15	$+ \{\text{favourite_id}, \text{favourited_at}\}$	
Fixed point	$\{\text{user_id}, \text{song_id}, \text{favourite_id}, \text{favourited_at}\}$	

$\{\{\text{user_id}, \text{song_id}\}\}^+ = \text{all } \textit{Favourites} \text{ attributes.} \Rightarrow \text{Composite alternate candidate key of } \textit{Favourites}.$

BCNF Violation Analysis on Universal Relation U

We now identify every FD in U whose left-hand side is not a superkey of U —these are the BCNF violations that drive the decomposition.

Key observation: No single attribute (or small set) determines all 38 attributes of U , so U has no finite candidate key without composing attributes from multiple entity groups. Every FD below therefore represents a violation in U .

#	Violating FD in U	Why it violates BCNF	Resolution
V1	$\text{user_id} \rightarrow$ all user attrs	user_id is not a superkey of U	Extract <i>Users</i>
V2	$\text{email} \rightarrow$ all user attrs	$\text{email} \not\supseteq U$	Extract <i>Users</i>
V3	$\text{song_id} \rightarrow$ all song attrs	$\text{song_id} \not\supseteq U$	Extract <i>Songs</i>
V4	$\text{file_path} \rightarrow$ all song attrs	$\text{file_path} \not\supseteq U$	Extract <i>Songs</i>
V5	$\text{album_id} \rightarrow$ all album attrs	$\text{album_id} \not\supseteq U$	Extract <i>Albums</i>
V6	$\text{album_title} \rightarrow$ all album attrs	$\text{album_title} \not\supseteq U$	Extract <i>Albums</i>
V7	$\text{artist_id} \rightarrow$ all artist attrs	$\text{artist_id} \not\supseteq U$	Extract <i>Artists</i>
V8	$\text{artist_name} \rightarrow$ all artist attrs	$\text{artist_name} \not\supseteq U$	Extract <i>Artists</i>
V9	$\text{genre_id} \rightarrow$ genre_name	$\text{genre_id} \not\supseteq U$	Extract <i>Genres</i>
V10	$\text{playback_id} \rightarrow$ all play attrs	$\text{playback_id} \not\supseteq U$	Extract <i>PlaybackHistory</i>
V11	$\text{playlist_id} \rightarrow$ all playlist attrs	$\text{playlist_id} \not\supseteq U$	Extract <i>Playlists</i>
V12	$\text{favourite_id} \rightarrow$ all fav attrs	$\text{favourite_id} \not\supseteq U$	Extract <i>Favourites</i>
V13	$\text{report_id} \rightarrow$ all report attrs	$\text{report_id} \not\supseteq U$	Extract <i>WeeklyReports</i>
V14	$\text{song_id} \rightarrow$ $\text{album_id}, \text{album_title}$	Partial transitive dep. in U	Separate <i>Songs/Albums</i>

Step-by-Step Decomposition to BCNF

Step 1 — Extract Users

Violation: $\text{user_id} \rightarrow \dots$ (V1) where user_id is not a superkey of U .

Compute $\{\text{user_id}\}^+$ restricted to U :

$\{\text{user_id}\}^+ = \{\text{user_id}, \text{email}, \text{username}, \text{password_hash}, \text{user_created_at}, \text{last_login}\}$

Decompose: $R_1 = \{\text{user_id}\}^+, \quad R_2 = U \setminus (\{\text{user_id}\}^+ \setminus \{\text{user_id}\})$

Resulting relation:

Users(user_id, email, username, password_hash, user_created_at, last_login)

FDs that hold: FD1, FD2, FD3 Candidate keys: $\{\text{user_id}\}, \{\text{email}\}, \{\text{username}\}$
BCNF: ✓

Step 2 — Extract Albums

Violation: $\text{album_id} \rightarrow \dots$ (V5) in the remainder.

$\{\text{album_id}\}^+ = \{\text{album_id}, \text{album_title}, \text{release_year}, \text{album_art_path}\}$

Resulting relation:

Albums(album_id, album_title, release_year, album_art_path)

FDs that hold: FD6, FD7 Candidate keys: $\{\text{album_id}\}, \{\text{album_title}\}$ BCNF: ✓

Step 3 — Extract Songs

Violation: $\text{song_id} \rightarrow \dots$ (V3) in the remainder.

$\{\text{song_id}\}^+ = \{\text{song_id}, \text{song_title}, \text{file_path}, \text{duration}, \text{file_format},$
 $\text{bit_rate}, \text{listen_count}, \text{track_num}, \text{album_id}\}$

Option 4: Using 'align*' with “ “latex

$\{\text{song_id}\}^+ = \{\text{song_id}, \text{song_title}, \text{file_path}, \text{duration}, \text{file_format},$
 $\text{bit_rate}, \text{listen_count}, \text{track_num}, \text{album_id}\}$

*Note: **album_id** is retained as a foreign key reference; Albums has already been extracted.*

Resulting relation:

Songs(song_id, song_title, file_path, duration, file_format,
bit_rate, listen_count, track_num, album_id)

FDs that hold: FD4, FD5 Candidate keys: $\{\text{song_id}\}, \{\text{file_path}\}$ BCNF: ✓

Check: Was FD18 ($song_id \rightarrow album_title$) eliminated?

In U , FD18 held because $song_id \xrightarrow{FD4} album_id \xrightarrow{FD6} album_title$ (transitivity).

After decomposition, `album_title` no longer appears in *Songs*; it lives only in *Albums*. The transitive dependency is broken by the decomposition. ✓

Step 4 — Extract Artists

Violation: $artist_id \rightarrow \dots$ (V7).

$$\{artist_id\}^+ = \{artist_id, artist_name, bio\}$$

Resulting relation:

Artists(artist_id, artist_name, bio)

FDs that hold: FD8, FD9 Candidate keys: {`artist_id`}, {`artist_name`} BCNF: ✓

Step 5 — Extract Genres

Violation: $genre_id \rightarrow genre_name$ (V9).

$$\{genre_id\}^+ = \{genre_id, genre_name\}$$

Resulting relation:

Genres(genre_id, genre_name)

FDs that hold: FD10, FD11 Candidate keys: {`genre_id`}, {`genre_name`} BCNF: ✓

Step 6 — Extract PlaybackHistory

Violation: $playback_id \rightarrow \dots$ (V10).

$$\{playback_id\}^+ = \{playback_id, user_id, song_id, played_at, \\ duration_played, completed, source\}$$

user_id and *song_id* are retained as foreign keys.

Resulting relation:

PlaybackHistory(playback_id, user_id, song_id, played_at, \\ duration_played, completed, source)

FDs that hold: FD12 Candidate key: {`playback_id`} BCNF: ✓

Step 7 — Extract Playlists

Violation: $\text{playlist_id} \rightarrow \dots$ (V11).

$$\{\text{playlist_id}\}^+ = \{\text{playlist_id}, \text{user_id}, \text{playlist_name}, \text{description}, \\ \text{is_public}, \text{playlist_created_at}\}$$

Resulting relation:

$$\text{Playlists}(\underline{\text{playlist_id}}, \text{user_id}, \text{playlist_name}, \text{description}, \\ \text{is_public}, \text{playlist_created_at})$$

FDs that hold: FD13 Candidate key: $\{\text{playlist_id}\}$ BCNF: ✓

Step 8 — Extract Favourites

Violation: $\text{favourite_id} \rightarrow \dots$ (V12), with a composite alternate key.

$$\{\text{favourite_id}\}^+ = \{\text{favourite_id}, \text{user_id}, \text{song_id}, \text{favourited_at}\}$$

Verify alternate key:

$$\{\text{user_id}, \text{song_id}\}^+ = \{\text{user_id}, \text{song_id}, \text{favourite_id}, \text{favourited_at}\} \supseteq \text{Favourites}$$

Resulting relation:

$$\text{Favourites}(\underline{\text{favourite_id}}, \text{user_id}, \text{song_id}, \text{favourited_at})$$

FDs that hold: FD14, FD15 Candidate keys: $\{\text{favourite_id}\}, \{\text{user_id}, \text{song_id}\}$
BCNF: ✓

Step 9 — Extract WeeklyReports

Violation: $\text{report_id} \rightarrow \dots$ (V13) with composite alternate key.

$$\{\text{report_id}\}^+ = \{\text{report_id}, \text{user_id}, \text{report_year}, \text{week_of_year}, \\ \text{total_songs_played}, \text{total_minutes}\}$$

Verify composite alternate key: $\{\{\text{user_id}, \text{report_year}, \text{week_of_year}\}\}^+ \supseteq \text{WeeklyReports}$
✓

Resulting relation:

$$\text{WeeklyReports}(\underline{\text{report_id}}, \text{user_id}, \text{report_year}, \text{week_of_year}, \\ \text{total_songs_played}, \text{total_minutes})$$

FDs that hold: FD16, FD17 Candidate keys: $\{\text{report_id}\}, \{\text{user_id}, \text{report_year}, \text{week_of_year}\}$
BCNF: ✓

Join / Associative Tables

The four JPA @ManyToMany join tables each hold exactly two foreign-key columns. We verify their BCNF status below.

Table	Attributes	FDs	BCNF?
<i>SongArtists</i>	song_id, artist_id	trivial only	✓
<i>SongGenres</i>	song_id, genre_id	trivial only	✓
<i>AlbumArtists</i>	album_id, artist_id	trivial only	✓
<i>PlaylistSongs</i>	playlist_id, song_id	trivial only	✓

A relation with only a composite primary key and no non-key attributes has no non-trivial FDs. The only FDs that hold are trivial (supersets of the key determine the key and all subsets). BCNF is therefore satisfied vacuously.

Final BCNF Schema

Complete Relation Listing

The final schema consists of 13 relations (9 entity tables + 4 join tables), all in BCNF.

Notation: Attribute = primary key; attribute = foreign key; plain = non-key attribute.

Relation	Schema & Candidate Keys
<i>Users</i>	(<u>user_id</u> , email, username, password_hash, user_created_at, last_login) CK: {user_id}, {email}, {username}
<i>Albums</i>	(<u>album_id</u> , album_title, release_year, album_art_path) CK: {album_id}, {album_title}
<i>Songs</i>	(<u>song_id</u> , song_title, file_path, duration, file_format, bit_rate, listen_count, track_num, album_id) CK: {song_id}, {file_path}
<i>Artists</i>	(<u>artist_id</u> , artist_name, bio) CK: {artist_id}, {artist_name}
<i>Genres</i>	(<u>genre_id</u> , genre_name) CK: {genre_id}, {genre_name}
<i>PlaybackHistory</i>	(<u>playback_id</u> , user_id, song_id, played_at, duration_played, completed, source) CK: {playback_id}
<i>Playlists</i>	(<u>playlist_id</u> , user_id, playlist_name, description, is_public, playlist_created_at) CK: {playlist_id}
<i>Favourites</i>	(<u>favourite_id</u> , user_id, song_id, favourited_at) CK: {favourite_id}, {user_id, song_id}
<i>WeeklyReports</i>	(<u>report_id</u> , user_id, report_year, week_of_year, total_songs_played, total_minutes) CK: {report_id}, {user_id, report_year, week_of_year}
<i>SongArtists</i>	(<u>song_id</u> , <u>artist_id</u>) CK: {song_id, artist_id}
<i>SongGenres</i>	(<u>song_id</u> , <u>genre_id</u>) CK: {song_id, genre_id}
<i>AlbumArtists</i>	(<u>album_id</u> , <u>artist_id</u>) CK: {album_id, artist_id}
<i>PlaylistSongs</i>	(<u>playlist_id</u> , <u>song_id</u>) CK: {playlist_id, song_id}

BCNF Verification Summary

Relation	Non-trivial FDs	All LHS are superkeys?	BCNF
<i>Users</i>	FD1, FD2, FD3	Yes (user_id, email, username)	✓
<i>Albums</i>	FD6, FD7	Yes (album_id, album_title)	✓
<i>Songs</i>	FD4, FD5	Yes (song_id, file_path)	✓
<i>Artists</i>	FD8, FD9	Yes (artist_id, artist_name)	✓
<i>Genres</i>	FD10, FD11	Yes (genre_id, genre_name)	✓
<i>PlaybackHistory</i>	FD12	Yes (playback_id)	✓
<i>Playlists</i>	FD13	Yes (playlist_id)	✓
<i>Favourites</i>	FD14, FD15	Yes (both candidate keys)	✓
<i>WeeklyReports</i>	FD16, FD17	Yes (both candidate keys)	✓
<i>SongArtists</i>	(none)	Vacuously true	✓
<i>SongGenres</i>	(none)	Vacuously true	✓
<i>AlbumArtists</i>	(none)	Vacuously true	✓
<i>PlaylistSongs</i>	(none)	Vacuously true	✓

Summary of Decomposition Steps

Step	Violating FD	Closure computed	New relation	BCNF
1	<code>user_id</code> $\rightarrow$ <code>user attrs</code>	$\{\text{user_id}\}^+ =$ <code>users</code>	<i>Users</i>	✓
2	<code>album_id</code> $\rightarrow$ <code>album attrs</code>	$\{\text{album_id}\}^+ =$ <code>albums</code>	<i>Albums</i>	✓
3	<code>song_id</code> $\rightarrow$ <code>song attrs</code>	$\{\text{song_id}\}^+ =$ <code>songs</code>	<i>Songs</i>	✓
4	<code>artist_id</code> $\rightarrow$ <code>artist attrs</code>	$\{\text{artist_id}\}^+ =$ <code>artists</code>	<i>Artists</i>	✓
5	<code>genre_id</code> $\rightarrow$ <code>genre_name</code>	$\{\text{genre_id}\}^+ =$ <code>genres</code>	<i>Genres</i>	✓
6	<code>playback_id</code> $\rightarrow$ <code>play attrs</code>	$\{\text{playback_id}\}^+ =$ <code>plays</code>	<i>PlaybackHistory</i>	✓
7	<code>playlist_id</code> $\rightarrow$ <code>pl attrs</code>	$\{\text{playlist_id}\}^+ =$ <code>plists</code>	<i>Playlists</i>	✓
8	<code>favourite_id</code> $\rightarrow$ <code>fav attrs</code>	$\{\text{favourite_id}\}^+ =$ <code>favs</code>	<i>Favourites</i>	✓
9	<code>report_id</code> $\rightarrow$ <code>rpt attrs</code>	$\{\text{report_id}\}^+ =$ <code>reports</code>	<i>WeeklyReports</i>	✓
—	(join tables)	trivial FDs only	4 join tables	✓

Conclusion. Starting from a 38-attribute Universal Relation U with 20 functional dependencies (14 base + 3 transitive + 3 composite-key FDs), 9 BCNF decomposition steps were applied, each justified by an explicit attribute closure computation. The resulting schema of 13 relations is entirely in BCNF: every non-trivial functional dependency in every relation has a superkey on its left-hand side. No redundancy of the form *repeated non-key data* remains, and all four join tables are in BCNF vacuously.

Notable Design Observations

Relations with Multiple Candidate Keys

Five relations contain more than one candidate key, reflecting real-world uniqueness constraints enforced by the application:

- **Users:** {`user_id`}, {`email`}, {`username`} — three candidate keys because users log in by either email or username, both declared `UNIQUE` in the DDL.
- **Albums:** {`album_id`}, {`album_title`} — deduplication during import relies on exact title match (`AlbumRepository.findByTitle`).
- **Songs:** {`song_id`}, {`file_path`} — the import guard queries by absolute path to prevent duplicate records.
- **Favourites:** {`favourite_id`}, {`user_id`, `song_id`} — the composite key enforces that a user can favourite a song at most once.
- **WeeklyReports:** {`report_id`}, {`user_id`, `report_year`, `week_of_year`} — the composite key prevents duplicate reports for the same user/week.

Transitive Dependencies Eliminated

The following transitive dependency chain was present in *U* and is eliminated by the decomposition:

$$\text{song_id} \xrightarrow{\text{FD4}} \text{album_id} \xrightarrow{\text{FD6}} \{\text{album_title}, \text{release_year}, \text{album_art_path}\}$$

After decomposition, *Songs* retains `album_id` as a foreign key only. The album attributes are exclusively in *Albums*, removing the transitive dependency entirely.

FD Preservation

BCNF decomposition does not guarantee preservation of every FD. In this schema, all FDs are preserved because each FD is entirely contained within a single extracted relation—no FD spans two different result relations. This is a consequence of the clean, surrogate-key-based design: each entity’s attributes are tightly grouped under its own primary key without cross-entity non-key determinants.

Lossless-Join Property

Each decomposition step produces a lossless join because the shared attribute (e.g., `album_id` between *Songs* and *Albums*) is a key of the extracted relation. By the lossless-join theorem (Heath, 1971), if $R = R_1 \bowtie R_2$ and the join attribute set is a key of R_1 , the decomposition is lossless. This holds for all 9 extraction steps above.